George Mason University

ECE 537: Image Processing and Computer Vision Spring 2026

Computer Exercise #1

Assigned: January 30, 2026
Due Date: February 09, 2026 (Midnight)

Introduction

Smartphone night photos often suffer from **underexposure**, **low contrast**, **color cast** (e.g. warm street lights), **noise** (high ISO), and **loss of detail**. In this lab you will build a classical (no deep learning) enhancement pipeline using **point processing** (tone/contrast) and **filtering** (denoising/sharpening), and justify each step.

Learning objectives

- Apply and compare point-processing operations: gamma, contrast stretch, CLAHE.
- Apply and compare filters: Gaussian, median, bilateral / NLM denoising.
- Understand and mitigate artifacts: noise amplification, halos, saturation clipping.
- Design and evaluate an end-to-end enhancement pipeline.

Deliverables Submit:

1) Code in a jupyter notebook that reproduces all outputs (python preferred).
2) A 2–4 page PDF report with figures and **answers to all questions**.
3) Output images: `original`, `tonemap`, `denoise`, `sharpen`, `final`.

Theory Refresher

This pe-lab reviews the key concepts needed to justify the design choices you will make in the night photo enhancement pipeline.

1. Point Processing as Nonlinear Mapping

Point processing applies a nonlinear transformation independently to each pixel:

$$g[n,m] = T\big(f[n,m]\big).$$

Gamma correction. Gamma correction is defined (after normalization to $[0,1]$) as

$$g = f^{\gamma}.$$

For $\gamma < 1$, dark intensities are expanded and bright intensities are compressed. This makes gamma correction especially effective for *underexposed images*, where most pixel values lie in the low-intensity range.

Key implication. Gamma correction is:

- Nonlinear,
- Memoryless,
- Capable of improving visibility,
- Can amplify noise in dark regions.

2. Histogram-Based Contrast Enhancement

Histogram equalization attempts to redistribute intensities to achieve a more uniform histogram.

Global histogram equalization. Global HE ignores spatial context and can:

- Over-amplify noise,
- Destroy brightness consistency,
- Produce unnatural contrast in night scenes.

CLAHE (Contrast-Limited Adaptive Histogram Equalization). CLAHE operates on local tiles and limits contrast amplification using a `clipLimit`.

- Small tiles $\Rightarrow$ strong local adaptation (risk of blockiness)
- Large tiles $\Rightarrow$ behavior closer to global HE
- Larger `clipLimit` (read documentation) $\Rightarrow$ stronger contrast and stronger noise amplification

3. Linear Filtering and Noise

Linear spatial filtering is defined by convolution:

$$g[n, m] = (f * h)[n, m].$$

Gaussian smoothing. Gaussian filters are linear, shift invariant, and separable. They suppress high spatial frequencies smoothly and do not introduce ringing, but they blur edges.

Median filtering. Median filtering is nonlinear and is particularly effective at removing impulse (salt-and-pepper) noise while preserving edges.

Bilateral filtering. We have not yet covered bilateral filtering, but the idea is fairly simple. The goal is to smooth an image while, at the same time, not blurring images. So, instead of a regular Gaussian filter that decays as a function of distance, $\|p-q\|$, it also decays as a function of the difference in image intensities at p and q so that if there is an edge (difference in intensities is large) then the Gaussian filter does little smoothing. The weights of the bilateral filter are given by

$$w(p, q) \propto \exp\left(-\frac{\|p-q\|^2}{2\sigma_s^2} - \frac{|I(p) - I(q)|^2}{2\sigma_r^2}\right).$$

The variances σ_s^2 and σ_r^2 are two that need to be specified. The bilinear filter is *not* linear or shift invariant. While it preserves edges, large parameter values can produce "cartoon-like" images.

4. Frequency-Domain Interpretation

Low-light noise is often dominated by high-frequency components. Contrast enhancement increases the gain applied to these frequencies.

- Smoothing filters act as low-pass filters, reducing noise but also attenuating edges.
- Sharpening (high-boost filtering) increases high-frequency energy and therefore must be applied carefully after denoising.

5. Unsharp Masking as High-Boost Filtering

Unsharp masking can be written as:

$$g = f + \alpha(f - G_\sigma * f).$$

This corresponds to a frequency response:

$$H(\omega) = 1 + \alpha\big(1 - G_\sigma(\omega)\big),$$

which amplifies mid-to-high frequencies.

Important caution. Because noise is also high frequency, unsharp masking:

- Must be applied after denoising,
- Should use moderate values of α,
- Can introduce halos and clipping artifacts.

6. Color Considerations

Independent processing of RGB channels often causes hue shifts. To avoid this:

- Perform tone and contrast operations on a luminance channel (Lab L or HSV V),
- Leave chromatic channels mostly unchanged.

Computer Exercise

1 Getting Started

The first step is to get one or more nighttime images to process (recommended: your own phone photo). It must include at least two of the following: deep shadows, bright lights, visible noise, color cast, haze.

A pipeline of what you will build is:

Step 1: White balance / color correction (optional but recommended)

Step 2: Tone lift (gamma) + contrast enhancement (CLAHE)

Step 3: Denoise (edge-preserving)

Step 4: Sharpen (unsharp mask) with clipping control

Step 5: Final mild contrast / saturation adjustment

2 Task 1: Baseline + Diagnostics (10%)

(a) Load the color image. If BGR, convert to RGB for display.

(b) Compute a grayscale version of the image (luminance proxy).

(c) Save a baseline montage: original RGB, grayscale, and an "exposure map" e.g. show pixels below some minimum and above some maximum,, e.g. 20 and 235, respectively). An exposure map is simply a diagnostic visualization that flags under-exposed (too dark) and over-exposed (too bright / saturated) pixels.

3 Task 2: Point Processing for Exposure and Contrast (30%)

Work in a way that avoids ruining color:

(a) Convert the image to **Lab** (or **HSV**) and apply tone/contrast changes to the **L** (or **V**) channel only.

(b) Implement and compare:

  (i) Gamma lift on luminance: $L' = 255\,(L/255)^{\gamma}$ with $\gamma \in \{0.4, 0.6, 0.8\}$.

  (ii) CLAHE on luminance with at least two parameter settings.

  (iii) Percentile contrast stretch on luminance (e.g. 1%–99%).

(c) Save a montage comparing at least 4 results.

Questions (answer in report)

1. Which method gives the best perceptual improvement *before denoising*? Why?
2. Explain how `clipLimit` and `tileGridSize` influence CLAHE artifacts.
3. Why is $\gamma < 1$ typically used for night enhancement?

Task 3: Denoising (25%)

Starting from your best Task 2 output, compare at least two denoisers:

(a) Bilateral filtering on luminance (or on each channel carefully)

(b) Non-local means denoising (`cv.fastNlMeansDenoisingColored`)

(c) (Optional) Median filter for salt-and-pepper artifacts

Save a montage: `denoise_compare.png`.

Questions (answer in report)

1. Why does contrast enhancement tend to make noise more visible?
2. Which denoiser preserved edges best on your image? Provide evidence (zoomed crop).

Task 4: Sharpening with Artifact Control (20%)

Implement unsharp masking:

$$\text{blur} = G_\sigma * f, \qquad g = f + \alpha(f - \text{blur}).$$

Try at least three (σ, α) pairs (e.g. $\sigma \in \{1, 2\}$, $\alpha \in \{0.6, 1.0, 1.4\}$). Save a grid and your final choice.

Questions (answer in report)

1. What happens as σ increases? (What spatial scales are boosted?)
2. What visible artifacts occur if α is too large?

Task 5: Objective Evaluation (15%)

Compute at least two metrics *before vs after*:

- **Sharpness proxy:** Variance of Laplacian

  A sharpness proxy is a computable scalar metric used to approximate perceived image sharpness (or edge acuity) without requiring human judgment. It is called a proxy because it does not measure sharpness directly, but instead quantifies a correlated signal property—typically high-frequency content, edge strength, or local contrast. In image processing and computer vision, sharpness proxies are used for autofocus, image quality assessment, deblurring evaluation, and optimization objectives. Here, you use a Laplacian-baed focus meausure

  $$S = \mathrm{Var}(\nabla^2 I)$$

  i.e., the variance of the Laplacian, which is a sum of second-order spatial derivatives.

- **Global contrast:** Standard deviation of luminance.
- **Shadow visibility:** The fraction of pixels below a threshold (e.g. < 20) before vs after.

Questions (answer in report)

1. Do the metrics agree with perceived improvement? Explain any mismatch.
2. Which metric is most informative for night enhancement?

Starter Code (OpenCV + Python)

Given below and downloadable from Canvas are most of the functions that you will need. Your submission should be complete, commented, and produce all required outputs.

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

def imread_bgr(path):
    img = cv.imread(path, cv.IMREAD_COLOR)
    if img is None:
        raise FileNotFoundError(f"Could not read: {path}")
    return img

def bgr2rgb(img):
    return cv.cvtColor(img, cv.COLOR_BGR2RGB)

def to_uint8(x):
    return np.clip(x, 0, 255).astype(np.uint8)

def gamma_u8(u8, gamma):
    x = u8.astype(np.float32) / 255.0
    y = np.power(x, gamma)
    return to_uint8(255.0 * y)

def clahe_u8(u8, clipLimit=2.0, tileGridSize=(8,8)):
    op = cv.createCLAHE(clipLimit=clipLimit, tileGridSize=tileGridSize)
    return op.apply(u8)

def contrast_stretch_percentile(u8, p_low=1, p_high=99):
    lo, hi = np.percentile(u8, [p_low, p_high])
    if hi <= lo + 1e-9:
        return u8.copy()
    out = (u8.astype(np.float32) - lo) * (255.0 / (hi - lo))
    return to_uint8(out)

def unsharp_mask_u8(u8, sigma=1.0, alpha=1.0):
    blur = cv.GaussianBlur(u8, (0,0), sigmaX=sigma, sigmaY=sigma)
    detail = cv.subtract(u8, blur)
    sharp = cv.addWeighted(u8, 1.0, detail, alpha, 0.0)
    return sharp

def var_laplacian(u8):
    lap = cv.Laplacian(u8, cv.CV_64F, ksize=3)
    return float(lap.var())

def global_contrast(u8):
    return float(u8.std())

def shadow_fraction(u8, thr=20):
    return float(np.mean(u8 < thr))

def show_rgb(img_bgr, title=""):
    plt.figure(figsize=(6,4))
    plt.imshow(bgr2rgb(img_bgr))
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
```

```
    plt.show()

def show_gray(u8, title=""):
    plt.figure(figsize=(6,4))
    plt.imshow(u8, cmap="gray", vmin=0, vmax=255)
    plt.title(title)
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```